

Analyse

Projet Ticketeer

2025/2026

membres de groupe :

- Sarah Badsì
- Anyas Meziani
- Sarah Abdelli

Table des matières

1. Acteurs du système.....	3
I. Voyageur.....	3
II. Agent de contrôle.....	4
III. Administrateur système.....	4
2. diagramme de cas d'utilisation.....	6
3. Diagramme de classe.....	7
4. Diagramme d'état du billet.....	8
5. Les scénarios.....	9
S1 --Achat billet (trajet direct).....	9
S2 — Achat billet (trajet avec correspondance).....	11
S2b — Aucun itinéraire disponible.....	13
S3 — Contrôle d'un billet valide (cas normal).....	14
S4 — Fraude : double scan.....	16
S5 — Billet hors itinéraire.....	18
S6 — Mauvaise date / heure / train.....	20
S7 — Billet invalide ou déjà utilisé (état global).....	24
S8 — Billet inconnu / UUID introuvable.....	27
S9 — Contrôle : billet introuvable (UUID inconnu).....	28
S10 — QR illisible / UUID invalide.....	30
.....	32
S11 — Token absent / agent non autorisé.....	33
S12 — Token expiré.....	34
S13 — Segment bon mais hors fenêtre (trop tôt / trop tard).....	35
S14 — Admin consulte l'historique d'un billet (traçabilité).....	36
S15 — Admin ajoute une ville et un segment (gestion réseau).....	37
6. Catalogue des questions et problèmes.....	39
I. Problèmes liés au cycle de vie du billet.....	39
II. Problèmes liés aux correspondances.....	39
III. Problèmes liés à la concurrence.....	40
IV. Problèmes liés à la sécurité.....	40
V. Problèmes liés à la validation contextuelle.....	41
VI. Problèmes liés à la traçabilité.....	41
VII. Problèmes liés au périmètre.....	41
VIII. Problèmes liés aux états invalides.....	41
IX. Problèmes liés à la performance.....	42
X. Problèmes liés aux données fixes.....	42

1. Acteurs du système

I. Voyageur

Rôle : utilisateur final qui achète et utilise un billet numérique.

Objectifs :

- trouver un trajet (direct ou avec correspondance)
- consulter l'itinéraire et les correspondances
- simuler le paiement
- obtenir un billet unique (QR Code)
- présenter ce billet lors du contrôle

Fonctionnalités utilisées (use cases) :

- Rechercher un trajet
- Consulter itinéraires (direct/correspondance)
- Sélectionner un itinéraire
- Simuler le paiement
- Générer billet numérique (UUID + QR)
- Afficher/consulter le billet

II. Agent de contrôle

Rôle : contrôleur à bord des trains, utilise l'application mobile pour scanner et valider les billets.

Objectifs :

- vérifier rapidement la validité du billet
- détecter les cas de fraude simples (billet réutilisé, hors trajet, mauvaise date)
- obtenir une réponse immédiate (≤ 2 secondes attendu)

Fonctionnalités utilisées (use cases) :

- S'authentifier (obtenir token)
- Scanner QR Code
- Valider billet (train/date/heure)
- Afficher résultat (valide / invalide / déjà utilisé / hors trajet)

- Enregistrer trace de contrôle

III. Administrateur système

Rôle : responsable des données (réseau, trains, clients) et de la supervision.

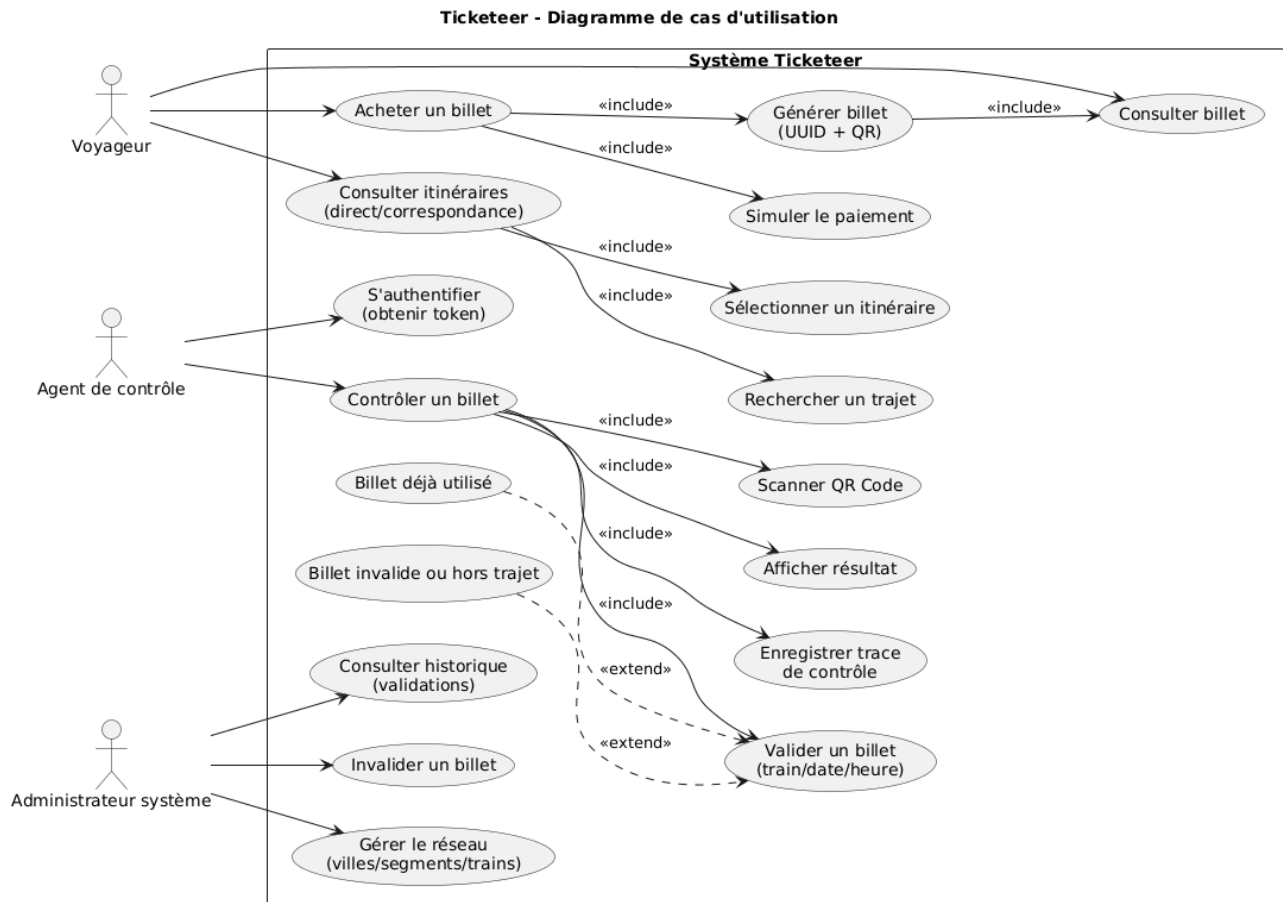
Objectifs :

- maintenir un réseau fixe (au moins 10 villes)
- gérer les trains/segments/horaire
- gérer les clients de test (au moins 4)
- consulter l'historique des validations pour traçabilité/analyse

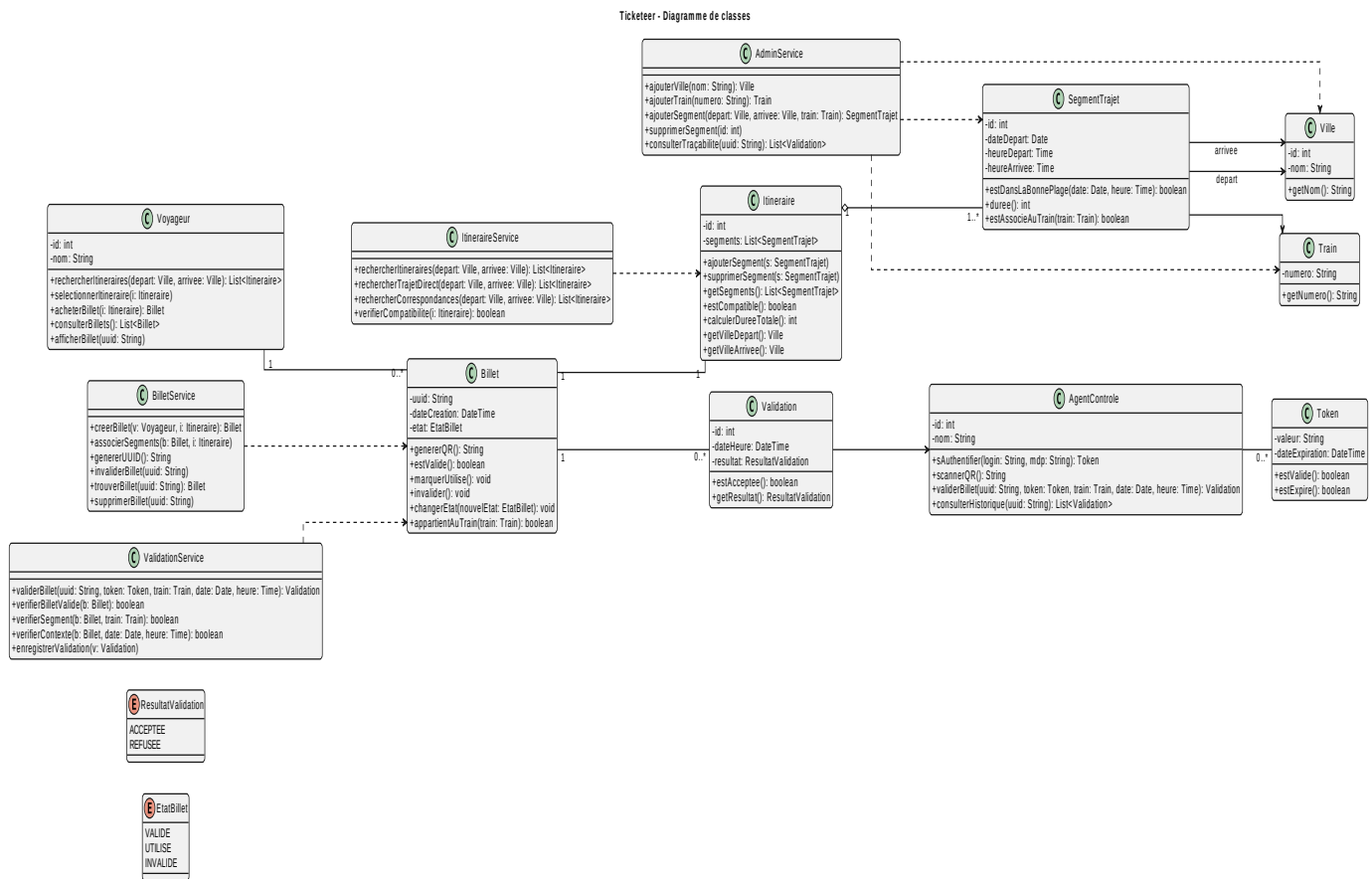
Fonctionnalités utilisées :

- Gérer trains / segments
- Gérer comptes clients
- Consulter traçabilité & historique validations

2. diagramme de cas d'utilisation



3. Diagramme de classe



Le diagramme de classes du système Ticketeer modélise l'architecture logique d'une application de gestion et de contrôle de billets numériques pour le transport ferroviaire. Le système repose sur plusieurs entités métier principales. La classe **Voyageur** représente l'utilisateur final ; elle lui permet de rechercher des itinéraires, de sélectionner un trajet et d'acheter un billet. Chaque voyageur peut posséder plusieurs billets. La classe **Billet** constitue l'élément central du système. Elle est identifiée par un UUID unique, possède une date de création et un état (VALIDE, UTILISE ou INVALIDE). Elle permet la génération d'un QR Code contenant uniquement l'UUID, ce qui garantit l'unicité et la traçabilité du billet. Chaque billet est associé à un seul **Itineraire**, et peut faire l'objet de plusieurs **Validation**, assurant ainsi l'historique des contrôles.

Un **Itineraire** est composé d'un ou plusieurs **SegmentTrajet**, ce qui permet de représenter aussi bien un trajet direct qu'un trajet avec correspondance. Chaque segment contient les informations relatives à la date et aux horaires, et est lié à une ville de départ, une ville d'arrivée et un train. Cette structure permet de vérifier qu'un billet est utilisé sur le bon train et au bon moment. Lors d'un contrôle, une instance de la classe **Validation** est créée pour enregistrer le résultat (ACCEPTEE ou REFUSEE) ainsi que la date et l'heure du contrôle. Cette validation est associée au billet et à un **AgentControle**, garantissant la traçabilité complète des opérations.

La sécurité du système repose sur la classe **Token**, générée lors de l'authentification d'un agent de contrôle. Ce token possède une date d'expiration et permet de sécuriser les opérations de validation. La logique métier est répartie dans plusieurs services : **ItineraireService** pour la recherche de

trajets et la gestion des correspondances, **BilletService** pour la création et la gestion des billets, **ValidationService** pour la vérification et l'enregistrement des contrôles, et **AdminService** pour l'administration des villes, trains et segments. L'ensemble de ces éléments garantit la cohérence des états, la sécurisation des contrôles, la gestion des correspondances et la traçabilité complète des validations, conformément aux exigences du système

4. Diagramme d'état du billet

Le diagramme d'état représente le cycle de vie d'un billet numérique au sein du système Ticketeer.

À sa création, suite à une simulation de paiement réussie, le billet est placé dans l'état VALIDE. Cet état signifie que le billet peut être présenté lors d'un contrôle et soumis à une validation contextuelle.

Un billet peut couvrir un ou plusieurs segments de trajet (cas d'un itinéraire avec correspondance). Lorsqu'une validation est acceptée :

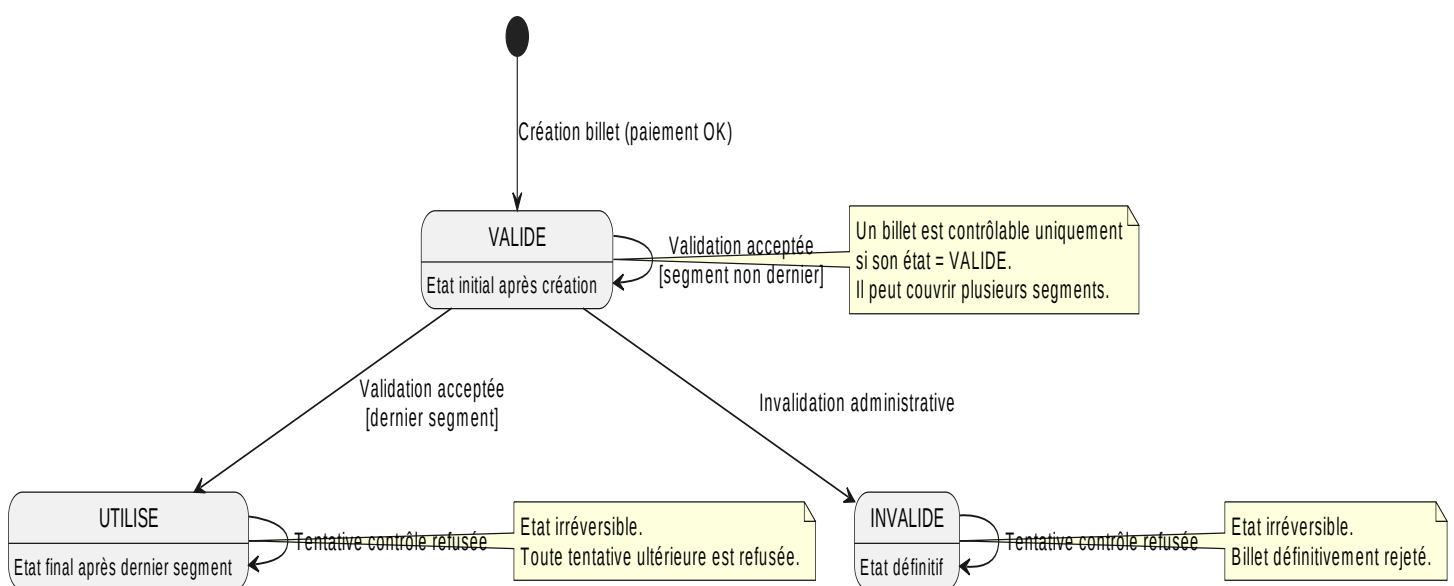
- si le segment contrôlé n'est pas le dernier du trajet, le billet reste dans l'état VALIDE, car d'autres segments restent à utiliser ;
- si le segment contrôlé correspond au dernier segment du trajet, le billet passe dans l'état UTILISÉ.

L'état UTILISÉ est irréversible : toute tentative ultérieure de validation est automatiquement refusée.

Le billet peut également être placé dans l'état INVALIDE, notamment en cas d'invalidation administrative ou de non-conformité détectée. Cet état est également irréversible.

Ainsi, le système garantit qu'un billet ne peut être utilisé qu'une seule fois pour l'ensemble de son itinéraire et qu'aucune réutilisation frauduleuse n'est possible

Diagramme d'état - Billet (Ticketeer)



5. Les scénarios

S1 --Achat billet (trajet direct)

Objectif

Permettre à un voyageur d'acheter un billet pour un trajet direct entre deux villes.

Logique détaillée

1. Le voyageur recherche un trajet (Ville A → Ville B).
2. Le système vérifie en base s'il existe un segment direct.
3. Le système affiche les horaires disponibles.
4. Le voyageur sélectionne un horaire et confirme l'achat (paiement simulé).
5. Le système :
 - Génère un **UUID unique**
 - Crée un billet avec état = **VALIDE**
 - Associe le billet au segment
6. Le billet est enregistré en base.
7. Un QR Code contenant uniquement l'UUID est généré.
8. Le voyageur reçoit son billet numérique.

Résultat attendu

Billet créé, traçable, prêt à être contrôlé

Préconditions

- Le réseau existe(villes + segments + horaires)
- Le système (services + base de données) fonctionne
- Le voyageur peut accéder à l'application

Remarque : si aucun trajet direct n'est trouvé, le scénario S2/S2b s'applique

Postconditions

- Un billet est créé en base
- Le billet a un UUID unique
- L'état du billet = **VALIDE**.
- Le billet est associé au voyageur.
- Un QR code (UUID seul) est généré.

- Le billet est affiché au voyageur.

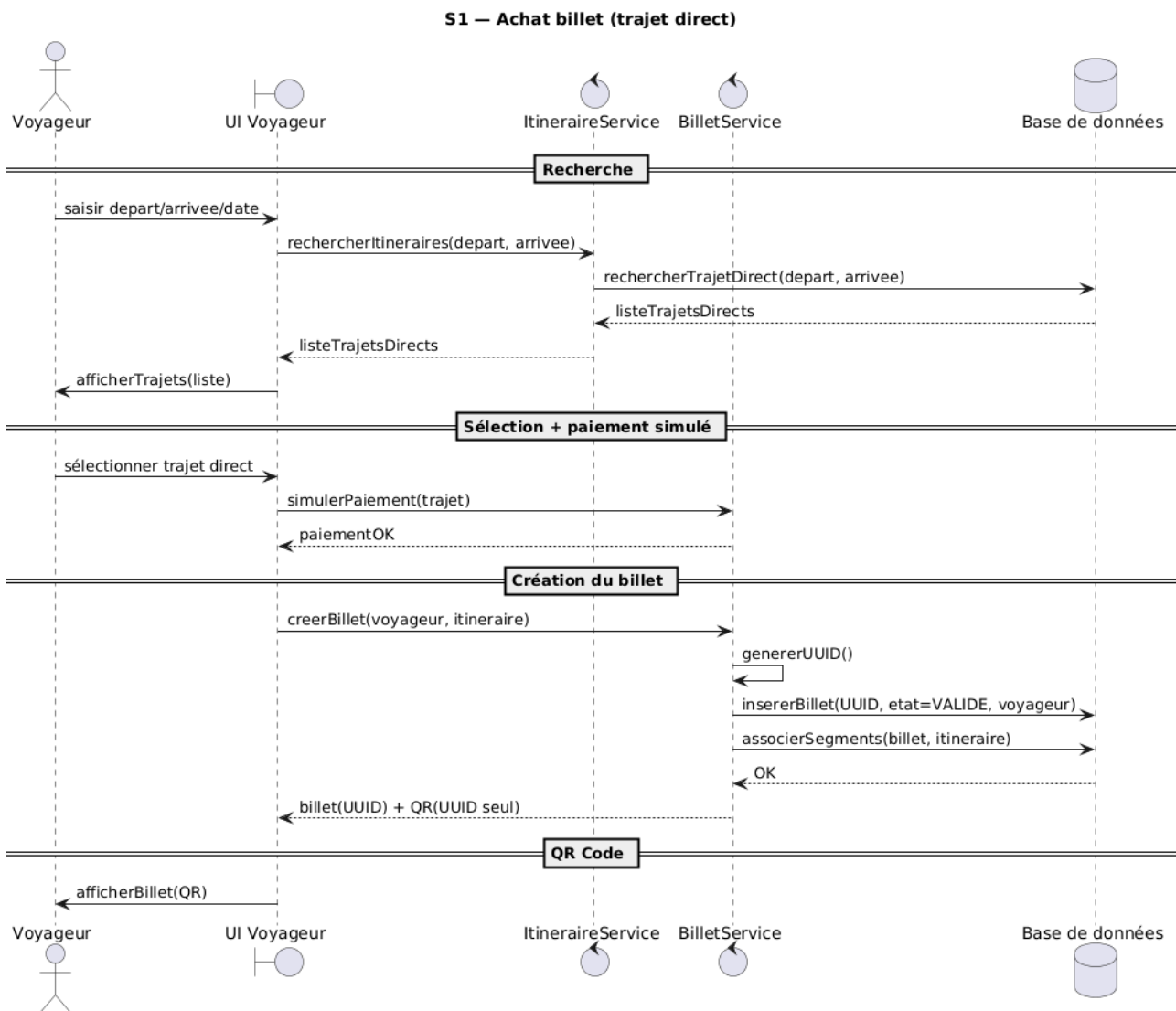
Invariants

Chaque billet a un UUID unique.

Le QR contient uniquement l'UUID.

Un billet a un seul état : VALIDE, UTILISE ou INVALIDE.

Un billet direct contient 1 seul segment.



S2 — Achat billet (trajet avec correspondance)

Objectif

Proposer un itinéraire multi-segments cohérent et générer un billet unique.

Logique détaillée

1. Le voyageur recherche un trajet $A \rightarrow C$.
2. Le système constate qu'il n'existe pas de segment direct.
3. Il cherche une combinaison possible (ex : $A \rightarrow B + B \rightarrow C$).
4. Il vérifie :
 - Compatibilité des horaires
 - Temps de correspondance suffisant
5. Il propose l'itinéraire complet.
6. Le voyageur confirme.
7. Le système :
 - Génère **un seul UUID**
 - Associe plusieurs segments au billet
 - État = **VALIDE**
8. Génération d'un QR unique.

Résultat

Un billet unique couvrant tous les segments

Préconditions

- Le réseau existe (villes + trains/segments).
- Aucun trajet direct n'est disponible (sinon S1 s'applique).
- Il existe au moins un itinéraire avec correspondance compatible (ordre + horaires).
- Le système fonctionne (services + base de données)

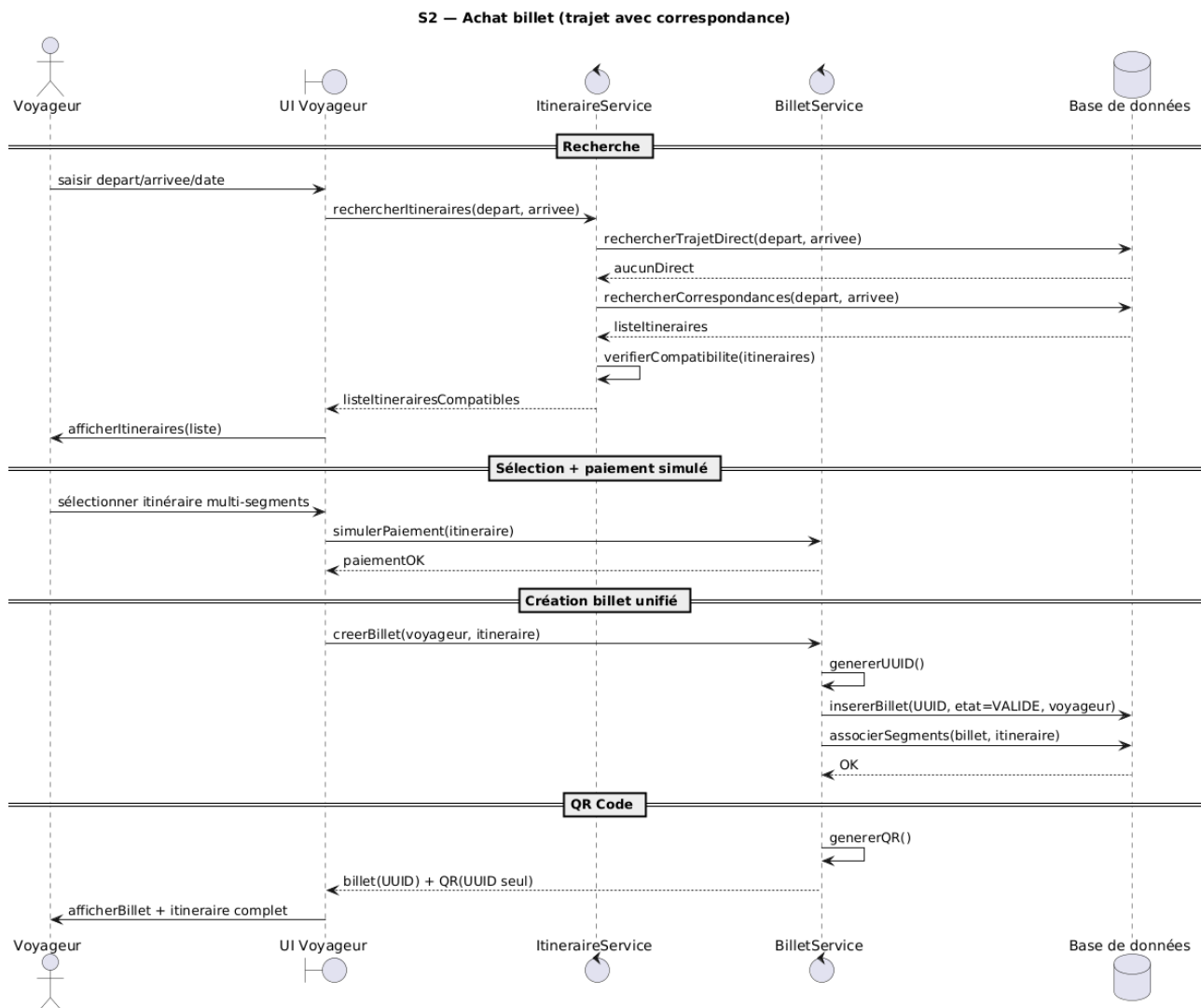
Postconditions

- Un billet est créé en base.
- Un UUID unique est généré.
- L'état du billet = **VALIDE**.
- Le billet est associé au voyageur et à plusieurs segments.
- Un QR code (UUID seul) est généré.

- Le billet et l'itinéraire complet sont affichés au voyageur

Invariants

- UUID toujours unique.
- QR contient uniquement l'UUID.
- Le billet a un seul état : VALIDE / UTILISÉ / INVALIDE.
- Un billet avec correspondance couvre plus d'un segment.
- Les segments de l'itinéraire sont compatibles (ordre + horaires).



S2b — Aucun itinéraire disponible

Objectif

Informer le voyageur qu'aucun itinéraire (direct ou avec correspondance) n'est disponible pour la recherche demandée

Logique

- Le voyageur saisit **départ, arrivée et date** dans l'application.
- Le système appelle **ItineraireService.rechercherItineraires(depart, arrivee)**.
- Le service tente :
 - la recherche de **trajets directs** (rechercherTrajetDirect),
 - puis la recherche de **correspondances** (rechercherCorrespondances).
- Aucun itinéraire compatible n'est trouvé (liste vide).
- L'application affiche le message : « **Aucun itinéraire disponible** »

Préconditions

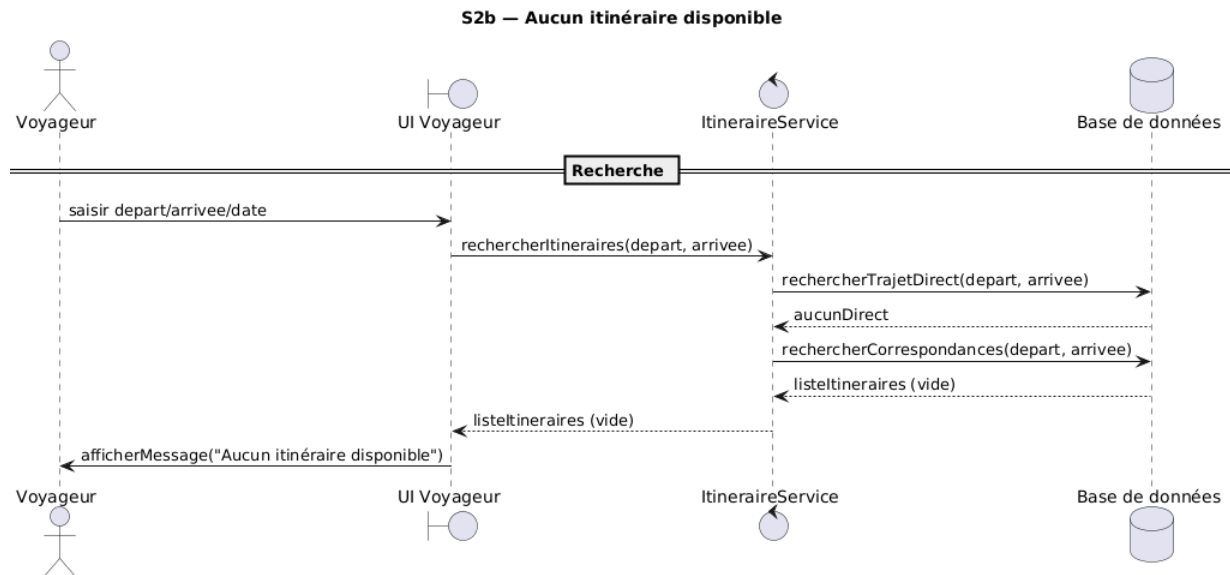
- Le réseau existe (villes, segments, trains).
- Le voyageur saisit un départ, une arrivée et une date valides.
- Aucun trajet direct **ni** itinéraire avec correspondance compatible n'existe.

Postconditions

- Aucun billet n'est créé (BilletService n'est pas appelé).
- Aucun UUID n'est généré.
- Aucun paiement n'est effectué.
- Un message d'échec est affiché : « **Aucun itinéraire disponible** »

Invariants

- Aucun billet ne doit être créé si aucun itinéraire n'existe.
- L'état du système reste inchangé (aucune création/modification de billet).
- La cohérence du réseau (villes/segments) est conservée



S3 — Contrôle d'un billet valide (cas normal)

Objectif

Valider un billet correctement lors d'un contrôle.

Logique détaillée

1. L'agent scanne le QR code.
2. L'application extrait l'UUID et envoie à **ValidationService** :
 - UUID,
 - token (agent authentifié),
 - numéro de train,
 - date et heure.
3. **ValidationService** vérifie :
 - que le billet existe,
 - que l'état du billet = VALIDE,
 - que le billet correspond au train,
 - que la date/heure du contrôle est cohérente.
4. Si tout est correct :
 - une **Validation** est enregistrée avec résultat = **ACCEPTEE**,
 - la réponse "Billet valide" est renvoyée.

Préconditions

- L'agent est authentifié (token disponible).
- Le billet existe en base.
- L'état du billet = VALIDE.
- Le billet correspond au train/date/heure du contrôle.

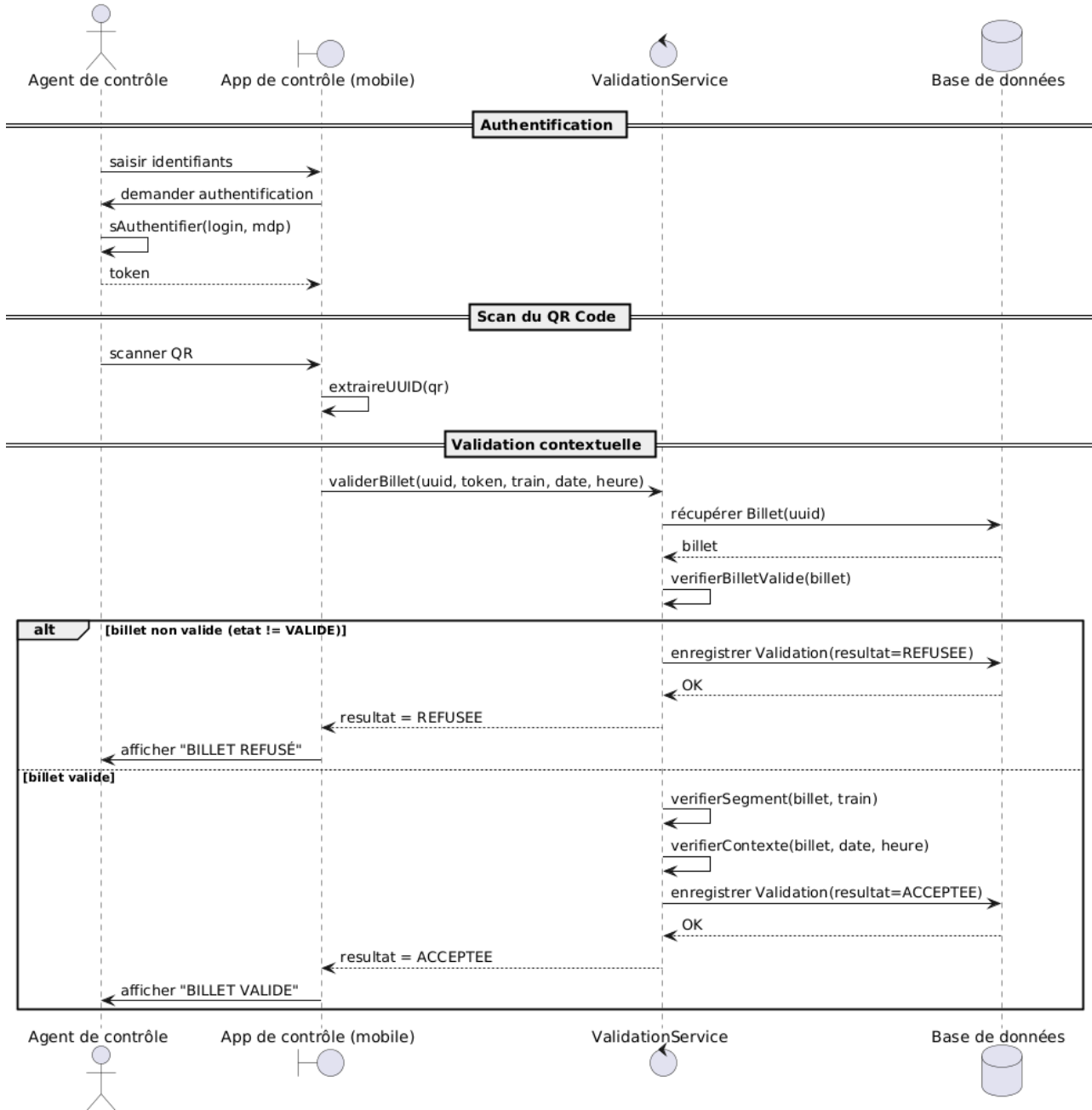
Postconditions

- Une validation est enregistrée (resultat = ACCEPTEE).
- Le résultat affiché = "BILLET VALIDE".

Invariants

- Une validation nécessite un agent authentifié.
- Un billet doit être VALIDE pour être accepté.
- Toute validation est enregistrée.
- Le QR contient uniquement l'UUID.

S3 — Contrôle d'un billet valide (cas nominal)



S4 — Fraude : double scan

Objectif

Empêcher la réutilisation du même billet.

Logique

1. Un premier contrôle a déjà eu lieu et le billet a été marqué **UTILISÉ**.
2. Un agent scanne à nouveau le QR code du même billet.
3. L'application extrait l'UUID et envoie à **ValidationService** :

4. UUID, token (agent authentifié), train, date, heure.
5. **ValidationService** charge le billet et constate que **l'état** $\neq$ **VALIDE** (état = UTILISE).
6. Le système refuse la validation et enregistre une **Validation** avec résultat = **REFUSEE**.
7. L'application affiche : « **Billet refusé / déjà utilisé** ».

Préconditions

- L'agent est authentifié (token disponible).
- Le billet existe en base.
- Le billet est déjà dans l'état **UTILISE**.

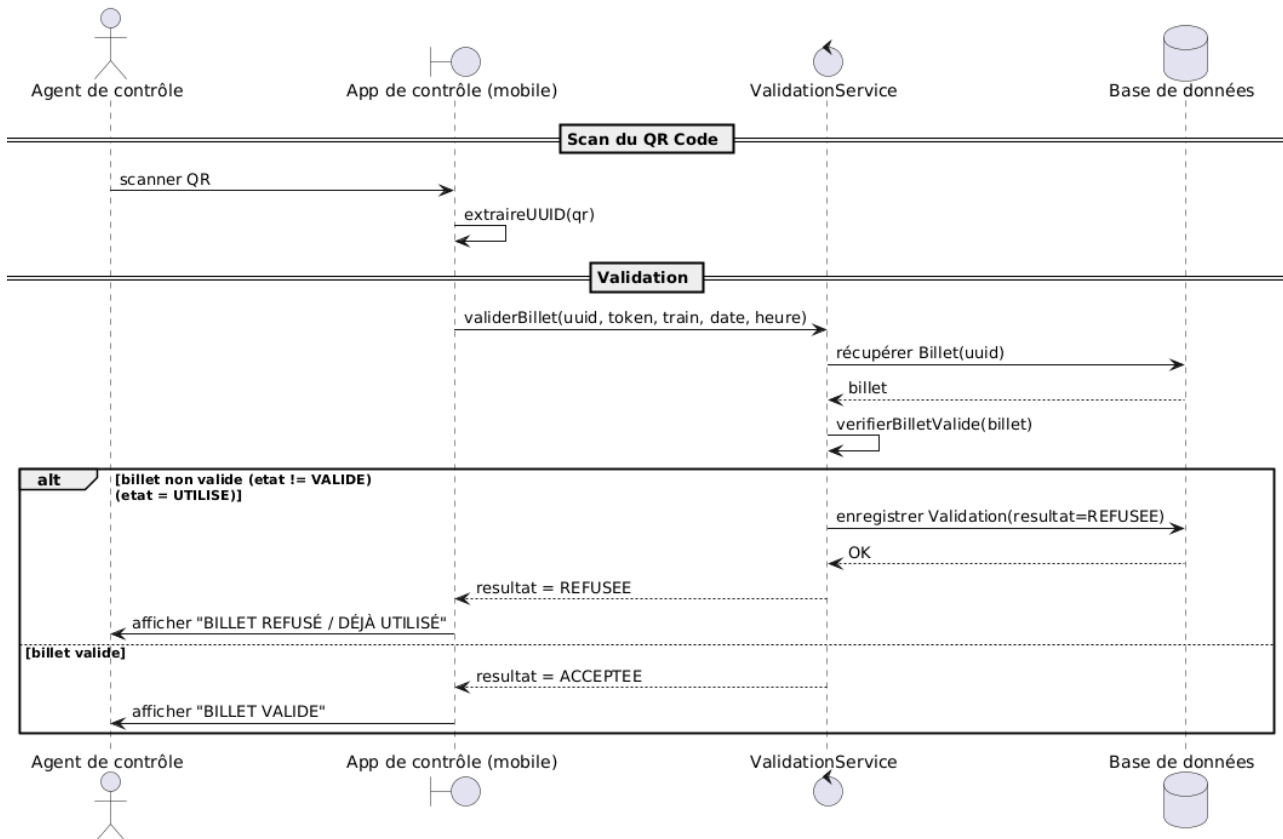
Postconditions

- Une validation est enregistrée avec résultat = **REFUSEE**.
- Le billet reste inchangé (toujours UTILISE).
- Le message affiché indique que le billet est refusé (déjà utilisé).

Invariants

- Un billet non VALIDE ne peut pas être accepté.
- Toute tentative de validation est enregistrée (ACCEPTEE ou REFUSEE).
- Le QR contient uniquement l'UUID.

S4 — Fraude : double scan (billet déjà utilisé)



S5 — Billet hors itinéraire

Objectif

Refuser la validation d'un billet lorsque celui-ci ne correspond pas au train contrôlé.

Logique détaillée

1. L'agent scanne le QR code.
2. L'application extrait l'UUID et appelle `ValidationService.validerBillet(uuid, token, train, date, heure)`.
3. `ValidationService` :
 - vérifie que le billet existe,
 - vérifie que l'état du billet est **VALIDE**,
 - vérifie que le billet correspond au train du contrôle (via la vérification de segment/train).
4. Si le train ne correspond pas à l'itinéraire du billet :
 - la validation est refusée (**ResultatValidation** = **REFUSEE**),
 - une **Validation** est enregistrée.
5. Sinon :

- la validation est acceptée (**ResultatValidation** = **ACCEPTEE**),
- une **Validation** est enregistrée.

Préconditions

- L'agent est authentifié (token disponible).
- Le QR code est lisible et contient un UUID valide.
- Le billet existe en base.

Postconditions

Si le train ne correspond pas

- Une validation est enregistrée avec résultat = **REFUSEE**.
- Le billet ne change pas d'état.
- Le message affiché indique un refus ("Hors itinéraire").

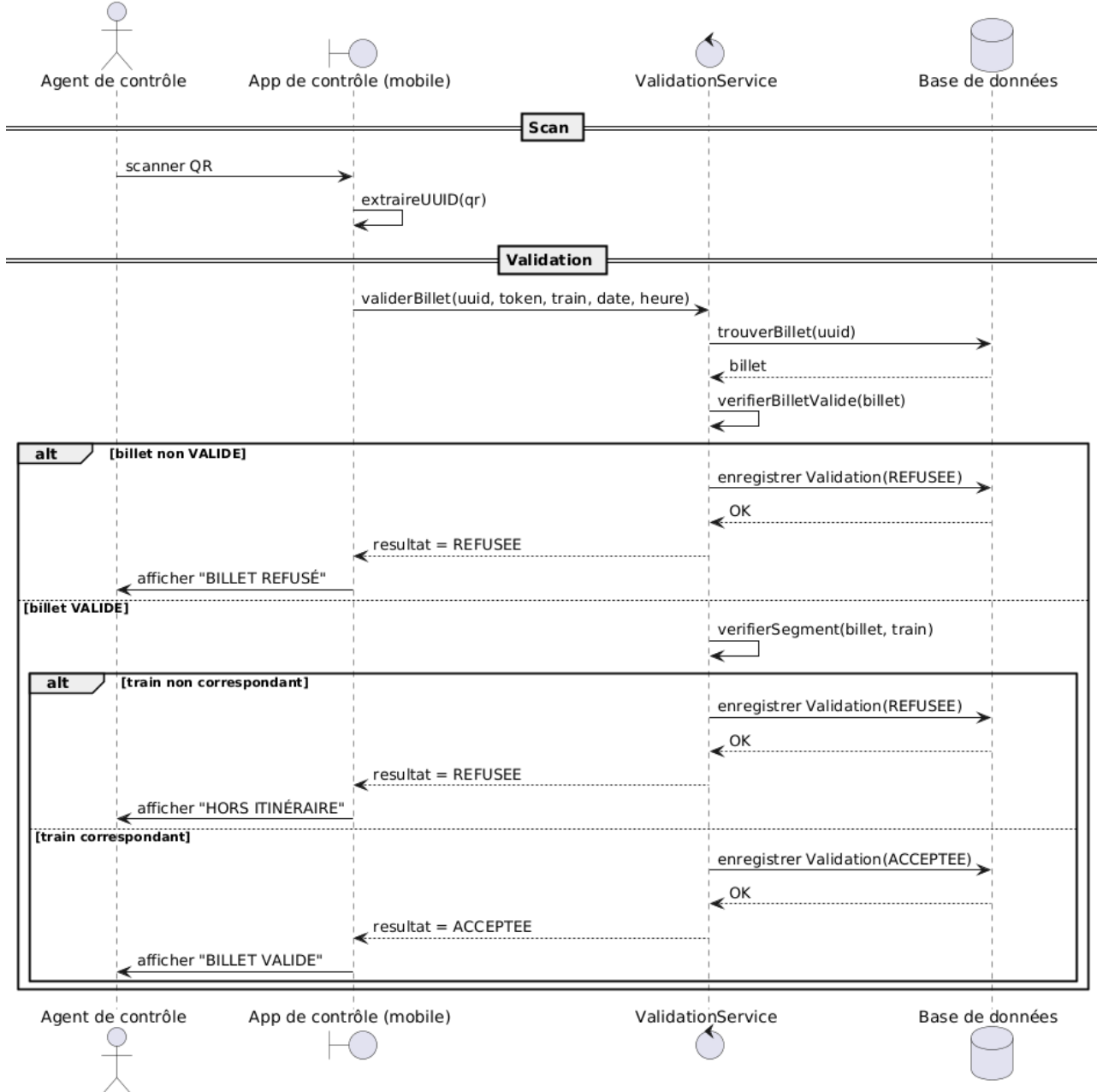
Si le train correspond

- Une validation est enregistrée avec résultat = **ACCEPTEE**.
- Le message affiché indique "Billet valide".

Invariants

- Un billet ne peut être accepté que sur un train correspondant à son itinéraire.
- Toute tentative de validation est enregistrée (ACCEPTEE ou REFUSEE).
- Un refus ne modifie pas l'état du billet.
- Le QR code contient uniquement l'UUID

S5 — Billet hors itinéraire / mauvais train



S6 — Mauvaise date / heure / train

Objectif

Refuser la validation d'un billet lorsque celui-ci est présenté pour une mauvaise date, une mauvaise heure ou sur un mauvais train, même si le segment correspond à l'itinéraire.

Logique

1. L'agent de contrôle scanne le QR code du billet.
2. Le système récupère le billet et son itinéraire.
3. Le système vérifie que :

- Le segment correspond à l'itinéraire.
- La date correspond à la date prévue.
- L'heure correspond au créneau autorisé.
- Le train correspond au train prévu.

4. Deux cas sont possibles :

- Si tous les paramètres correspondent → validation acceptée.
- Si au moins un paramètre (date, heure ou train) est incorrect → validation refusée.
-

Préconditions

- L'agent de contrôle est authentifié.
- Le billet existe.
- Le QR code est valide.
- Le billet est dans un état valide (non utilisé, non invalide).

Postconditions

Cas 1 — Date / heure / train incorrect

- La validation est refusée.
- Message retourné :
 - "Mauvaise date" ou
 - "Mauvaise heure" ou
 - "Mauvais train" (selon le cas).
- L'état du billet ne change pas.
- Une trace de la tentative de validation est enregistrée.

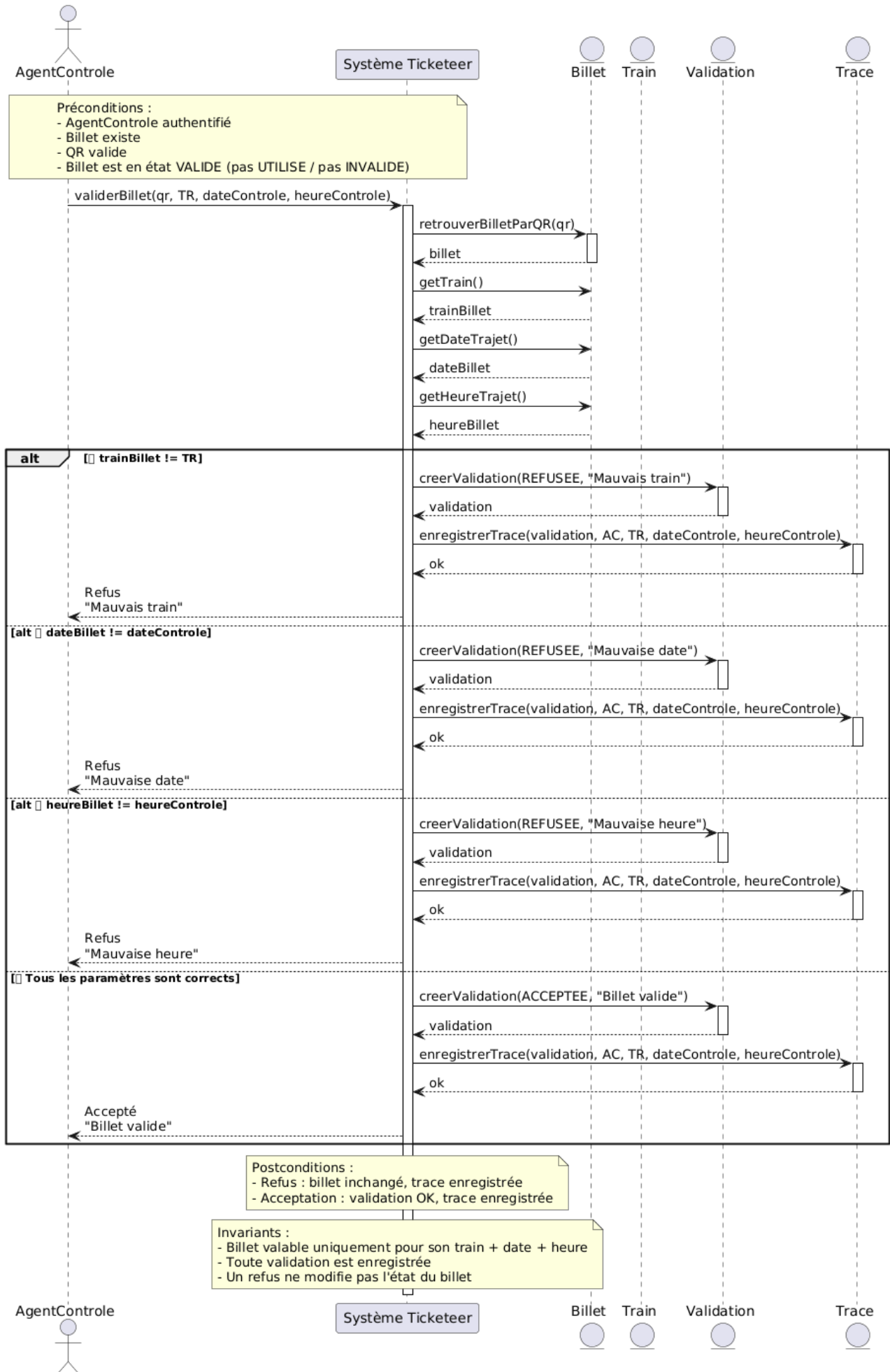
Cas 2 — Tous les paramètres correspondent

- La validation est acceptée.
- Message retourné : "**Billet valide**".
- Une trace est enregistrée.
- Le billet peut passer à l'état *utilisé* (si c'est la première validation).

Invariants

- Un billet n'est valable que pour la date, l'heure et le train définis lors de l'achat.
- Toute tentative de validation est enregistrée.
- Un refus ne modifie jamais l'état du billet.
- Un billet utilisé ne peut pas être validé une seconde fois.
- La cohérence entre billet, segment, train, date et heure doit toujours être respectée.

S6 — Mauvaise date / heure / train



S7 — Billet invalide ou déjà utilisé (état global)

Objectif

Refuser la validation d'un billet si son **état global** indique qu'il est **invalide** ou **déjà utilisé**, avant même de vérifier l'itinéraire, la date ou le train.

Logique

1. L'agent de contrôle scanne le QR code du billet.
2. Le système récupère le billet correspondant.
3. Le système lit l'état actuel du billet (**valide** / **utilisé** / **invalide**).
4. Deux cas de refus sont possibles :
 - Si l'état est **INVALIDE** → refus immédiat.
 - Si l'état est **UTILISÉ** → refus immédiat.
5. Si l'état est **VALIDE**, S7 ne s'applique pas : le système continue vers les vérifications S5/S6.

Préconditions

- L'agent de contrôle est authentifié.
- Le billet existe.
- Le QR code est valide (lisible et reconnu).

Postconditions

Cas 1 — Billet INVALIDE

- La validation est refusée.
- Message : "**Billet invalide**".
- L'état du billet ne change pas.
- Une trace est enregistrée.

Cas 2 — Billet déjà UTILISÉ

- La validation est refusée.
- Message : "**Billet déjà utilisé**".
- L'état du billet ne change pas.
- Une trace est enregistrée.

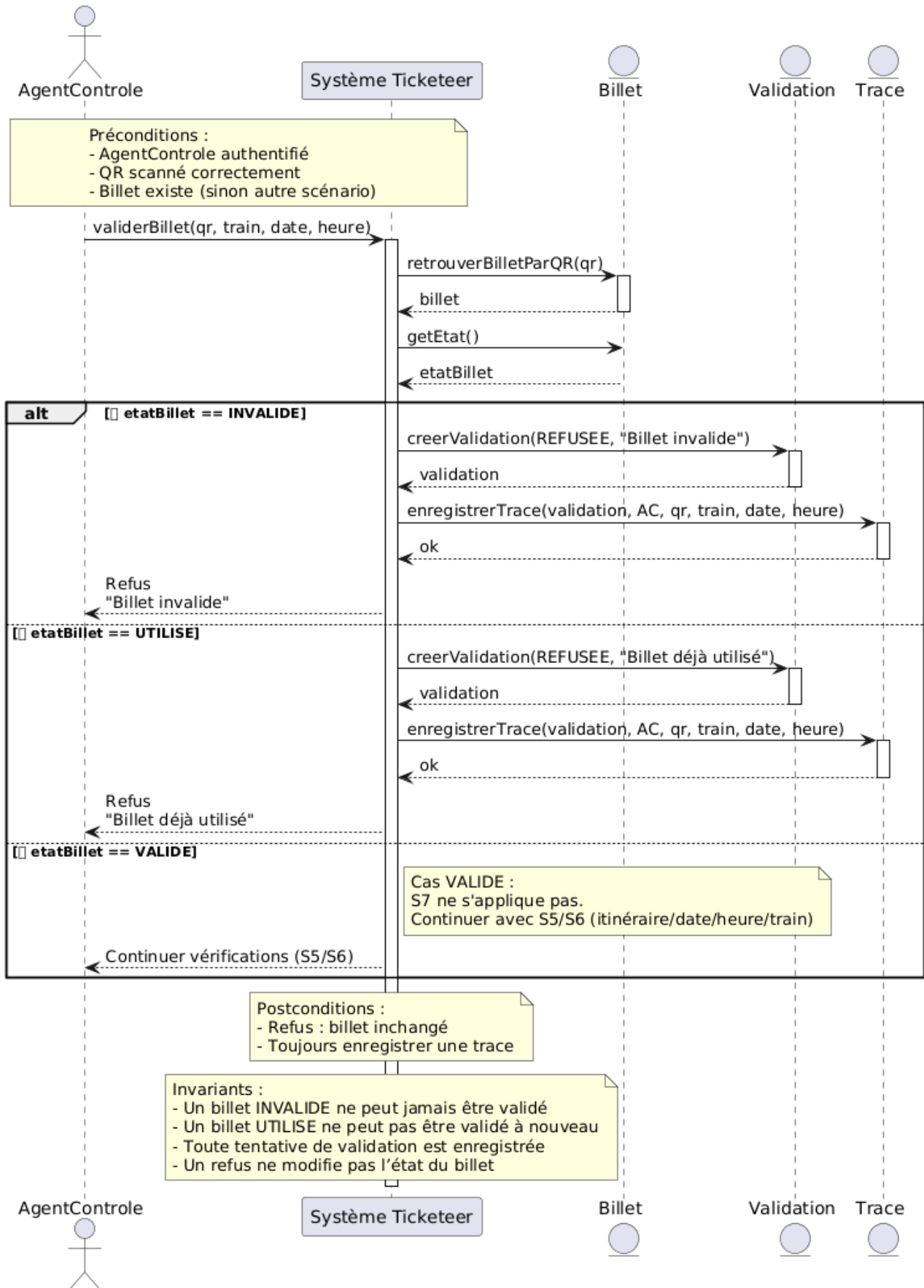
Cas 3 — Billet VALIDE

- S7 ne s'applique pas.
- Le système continue les contrôles (itinéraire / date / heure / train).

Invariants

- Un billet **INVALIDE** ne peut jamais être validé.
- Un billet **UTILISÉ** ne peut pas être validé une seconde fois.
- Toute tentative de validation (acceptée ou refusée) est enregistrée.
- Un refus ne modifie jamais l'état du billet.

S7 — Billet invalide ou déjà utilisé (état global)



S8 — Billet inconnu / UUID introuvable

Objectif

Refuser la validation lorsqu'un QR/UUID scanné ne correspond à **aucun billet** enregistré dans le système.

Logique

1. L'agent de contrôle scanne le QR code (UUID).
2. Le système tente de retrouver le billet associé à cet UUID.
3. Deux cas sont possibles :
 - Si aucun billet n'est trouvé → refus immédiat avec message "**Billet inconnu**".
 - Si le billet est trouvé → S8 ne s'applique plus, le système continue les vérifications (S7 puis S5/S6).

Préconditions

- L'agent de contrôle est authentifié.
- Un UUID/QR est fourni (scanné).

Postconditions

Cas 1 — UUID introuvable

- La validation est refusée.
- Message retourné : "**Billet inconnu**".
- Aucun état de billet ne change (car billet inexistant).
- Une trace de la tentative est enregistrée.

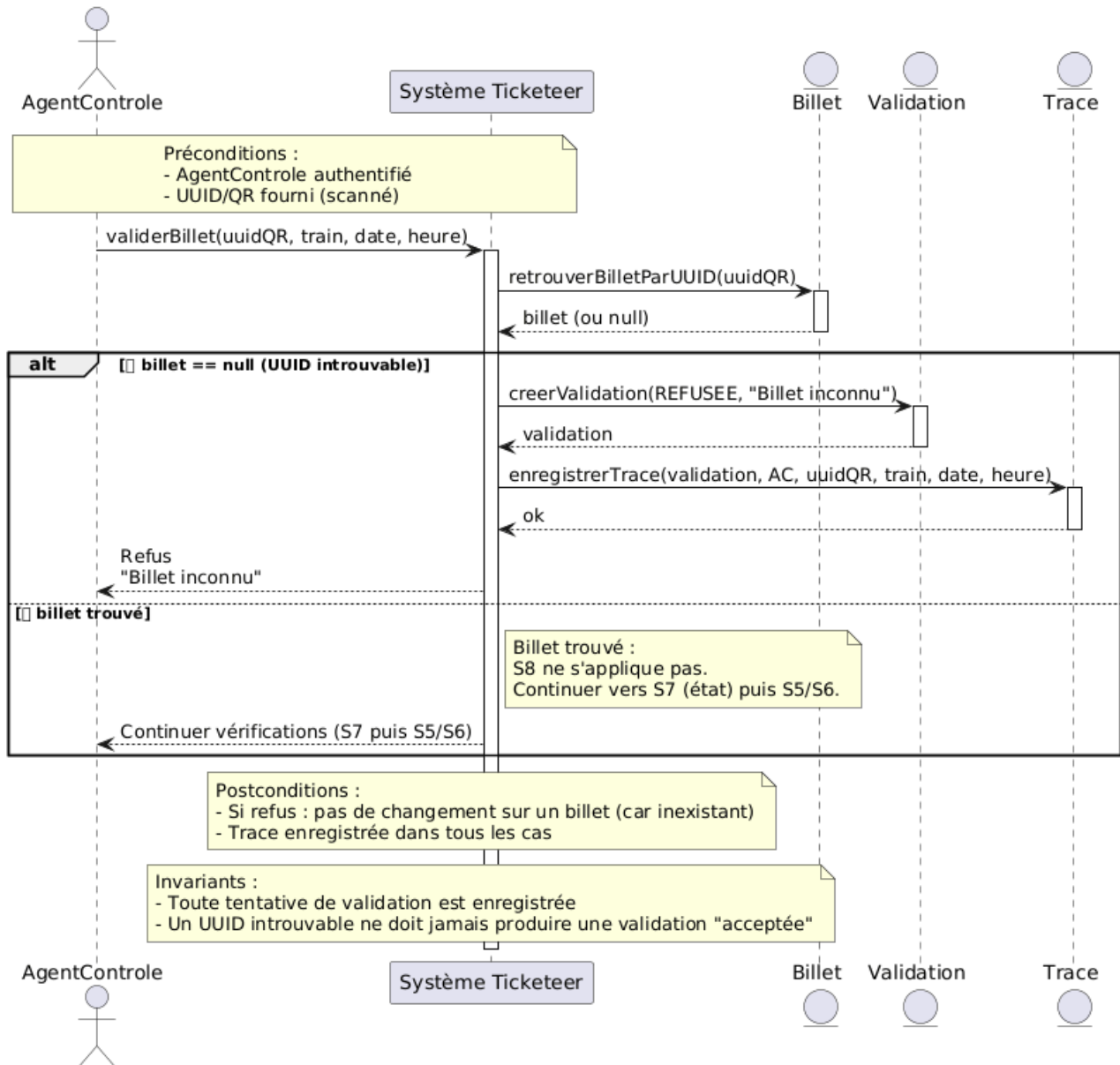
Cas 2 — UUID trouvé

- S8 ne s'applique pas.
- Le système continue la validation normale (état du billet, itinéraire, date/heure/train).
- Une trace pourra être enregistrée dans les scénarios suivants (selon ta règle métier, tu peux aussi tracer ici).

Invariants

- Toute tentative de validation est enregistrée.
- Un UUID introuvable ne peut jamais produire une validation acceptée.
- Le système ne doit jamais "inventer" un billet : sans billet trouvé, la validation est forcément refusée.

S8 — Billet inconnu / UUID introuvable



S9 — Contrôle : billet introuvable (UUID inconnu)

Objectif

Refuser le contrôle lorsque l'UUID du billet scanné ne correspond à **aucun billet** dans le système.

Logique

1. L'agent de contrôle scanne le QR code (UUID).
2. Le système tente de retrouver le billet correspondant.
3. Deux cas possibles :
 - Si aucun billet n'est trouvé → refus immédiat (**billet introuvable**).

- Si un billet est trouvé → S9 ne s'applique pas, on continue avec les vérifications suivantes (état, itinéraire, date/heure/train).

Préconditions

- L'agent est authentifié.
- Le QR/UUID est scanné correctement et envoyé au serveur.

Postconditions

Cas 1 — Billet introuvable (UUID inconnu)

- Le contrôle est refusé.
- Message : "**Billet introuvable**".
- Aucun état de billet ne change (puisque le billet n'existe pas).
- Une trace de tentative de contrôle est enregistrée.

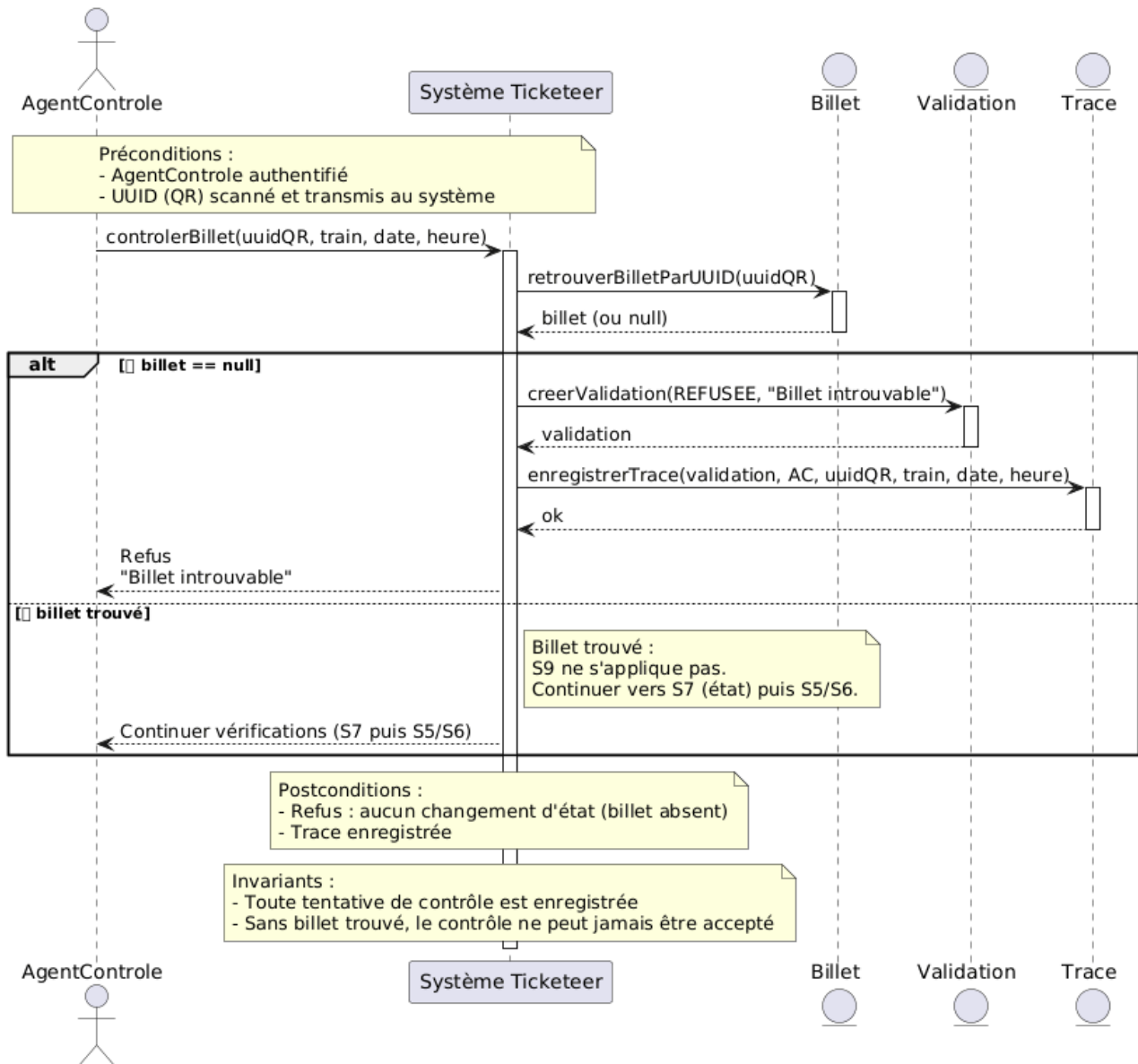
Cas 2 — Billet trouvé

- S9 ne s'applique pas.
- Le système continue le contrôle (S7 puis S5/S6).
- Les traces/validations seront enregistrées selon les scénarios suivants.

Invariants

- Toute tentative de contrôle est enregistrée.
- Un UUID inconnu ne peut jamais donner une validation acceptée.
- Le système ne doit jamais créer/modifier un billet à partir d'un UUID introuvable.

S9 — Contrôle : billet introuvable (UUID inconnu)



S10 — QR illisible / UUID invalide

Objectif

Refuser le contrôle si le QR code ne peut pas être lu correctement ou si l'UUID extrait est invalide (format incorrect, vide, incomplet).

Logique

1. L'agent de contrôle scanne le QR code.
2. Le système tente d'extraire l'UUID contenu dans le QR.
3. Le système vérifie le format de l'UUID (ex : structure UUID attendue).
4. Deux cas possibles :

- Si le QR est illisible ou l'UUID est invalide → refus immédiat.
- Si l'UUID est valide → le système peut continuer vers la recherche du billet (scénarios suivants).

Préconditions

- L'agent de contrôle est authentifié.
- Un QR a été scanné (même si son contenu est incorrect).

Postconditions

Cas 1 — QR illisible / UUID invalide

- Le contrôle est refusé.
- Message retourné : "**QR illisible / UUID invalide**".
- Aucun billet n'est modifié (car aucun billet ne peut être identifié).
- Une trace de la tentative est enregistrée.

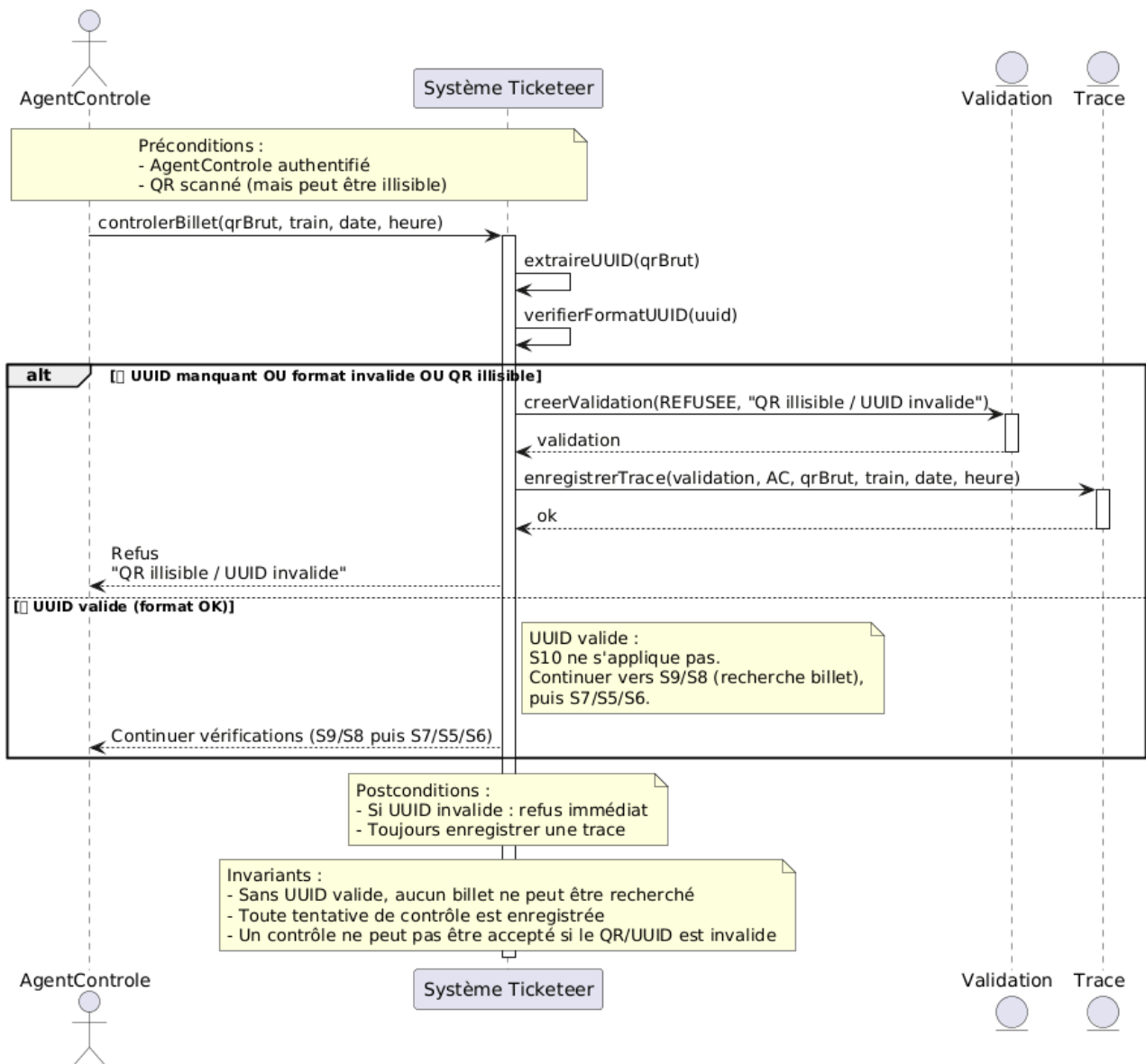
Cas 2 — UUID valide

- S10 ne s'applique pas.
- Le système continue le contrôle normal (recherche billet, état du billet, itinéraire, date/heure/train...).

Invariants

- Un contrôle ne peut pas être accepté si l'identifiant du billet (UUID) n'est pas valide.
- Toute tentative de contrôle (même échouée) est enregistrée.
- Tant que l'UUID n'est pas valide, aucune recherche de billet ne doit être effectuée.
- Le système doit empêcher toute validation "par défaut" sans identification fiable du billet.

S10 – QR illisible / UUID invalide



S11 — Token absent / agent non autorisé

Objectif

Refuser un contrôle si l'agent n'a pas de token (pas authentifié).

Logique

- L'agent scanne un QR.
- L'app appelle `ValidationService.validerBillet(...)` avec `token = NULL`.
- Le serveur refuse et enregistre une validation refusée.

Préconditions

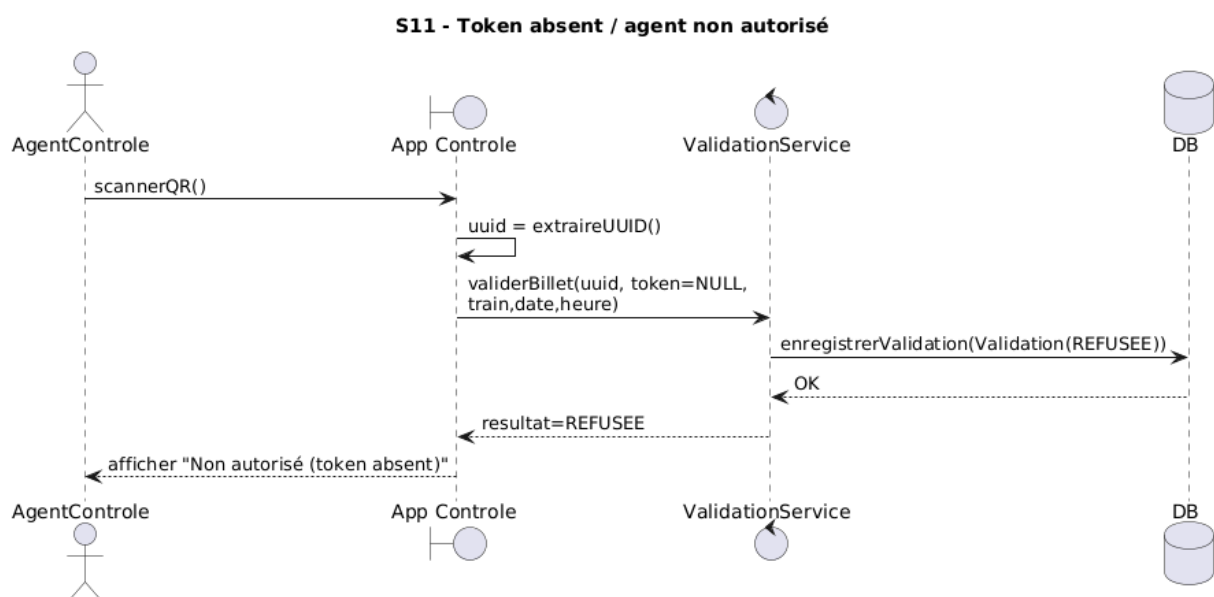
- QR scanné → UUID extrait.
- Token **absent** (NULL).

Postconditions

- Résultat = **REFUSEE**
- Une **Validation** est créée et enregistrée
- Aucune modification du **Billet**.

Invariants

- Aucune validation n'est acceptée sans token.
- Toute tentative est enregistrée (Validation créée).
- Le billet ne change pas si refus.



S12 — Token expiré

Objectif

Refuser le contrôle si le token est expiré.

Logique

- L'agent a un token.
- `Token.estExpire()` retourne vrai.
- Le serveur refuse et trace le refus.

Préconditions

- Token fourni.
- Token expiré (`token.estExpire() = true`).
- QR scanné.

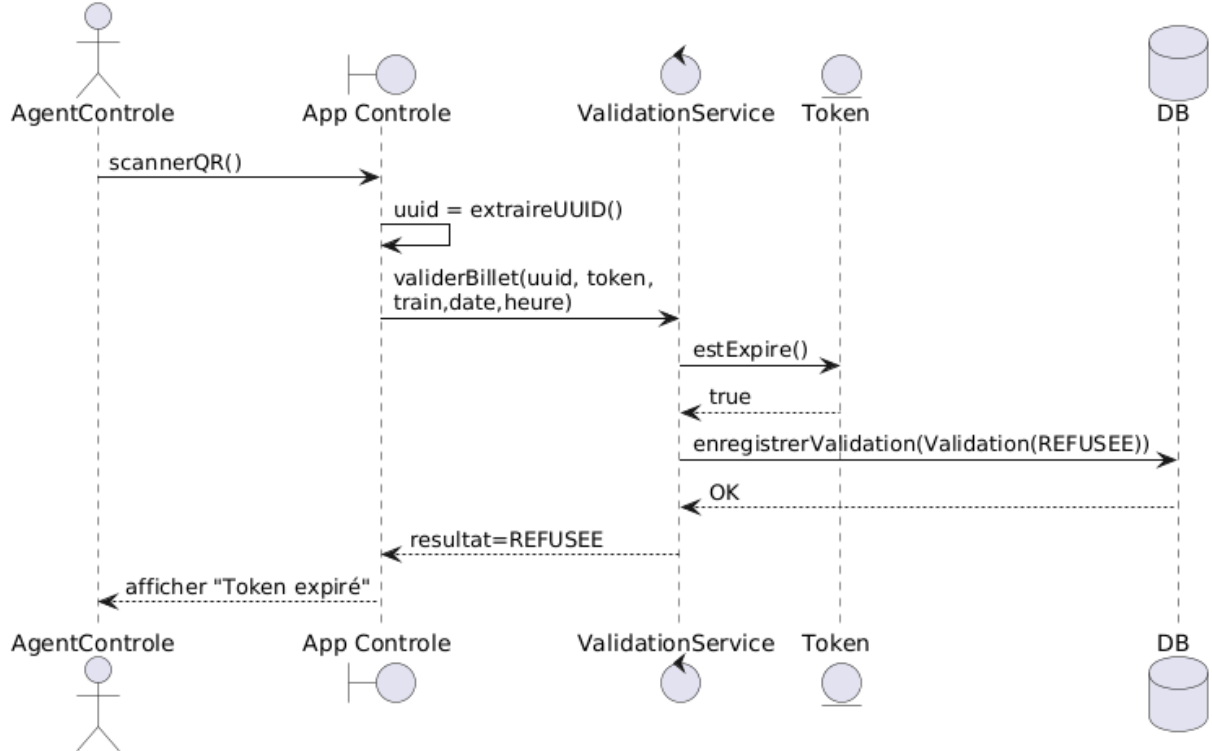
Postconditions

- Résultat = **REFUSEE**
- Une `Validation` est enregistrée.
- Aucun changement sur le billet.

Invariants

- Un token expiré ne permet jamais une validation acceptée.
- Toute tentative est enregistrée.
- Refus $\Rightarrow$ billet inchangé

S12 - Token expiré



S13 — Segment bon mais hors fenêtre (trop tôt / trop tard)

Objectif

Refuser si le billet est utilisé sur le bon segment, mais au mauvais moment.

Logique

- Le billet existe et est valide.
- Le segment est correct.
- `SegmentTrajet.estDansLaBonnePlage(date, heure)` retourne faux.
- Refus + trace.

Préconditions

- Token valide.
- Billet existe et est VALIDE.
- Segment correspond au train.
- Heure du contrôle hors plage.

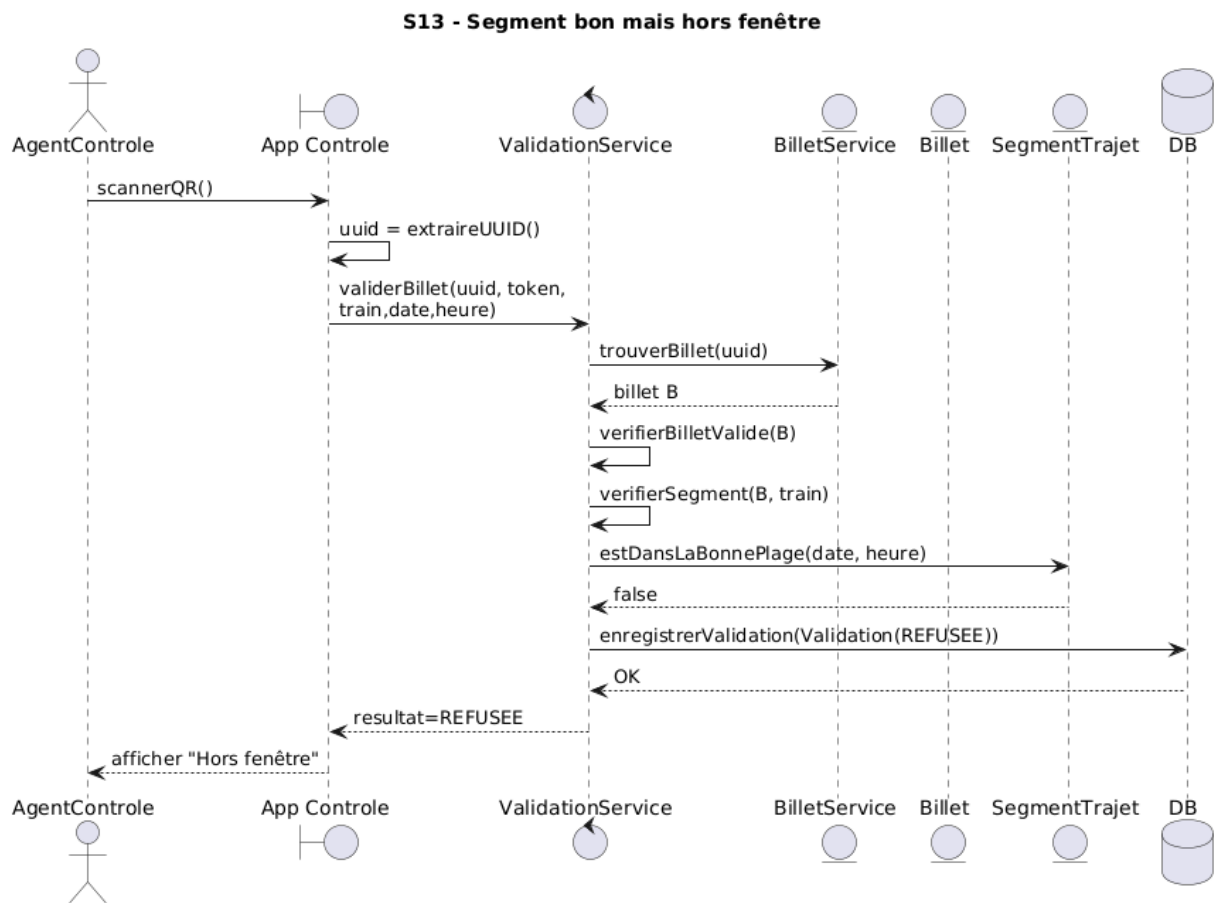
Postconditions

- Résultat = **REFUSEE**

- Validation enregistrée.
- Segment NON utilisé.
- État billet inchangé.

Invariants

- Un segment ne peut être validé que dans la bonne fenêtre horaire.
- Toute tentative est tracée.
- Refus $\Rightarrow$ pas de changement état billet.



S14 — Admin consulte l'historique d'un billet (traçabilité)

Objectif

Afficher la liste des validations d'un billet.

Logique

- Admin appelle `AdminService.consulterTraçabilite(uuid)`.
- Le système récupère les validations liées au billet.
- Affiche la liste.

Préconditions

- Admin autorisé.
- Le billet existe.

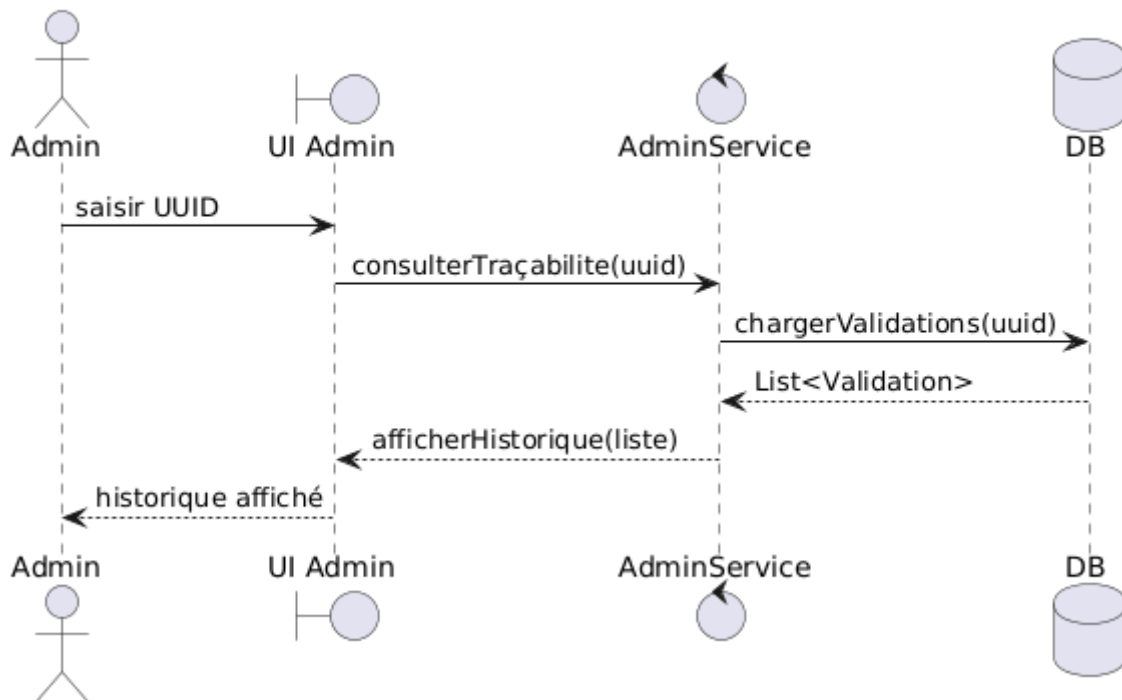
Postconditions

- Historique affiché (liste de Validation).
- Aucune modification sur le billet.

Invariants

- La consultation ne modifie pas les données.
- L'historique doit contenir toutes les validations existantes

S14 - Admin consulte l'historique d'un billet



S15 — Admin ajoute une ville et un segment (gestion réseau)

Objectif

Permettre à l'admin de maintenir le réseau : créer une ville + un segment.

Logique

1. Admin ajoute une ville : `AdminService.ajouterVille(nom)`
2. Admin ajoute un segment : `AdminService.ajouterSegment(depart, arrivee, train)`

Préconditions

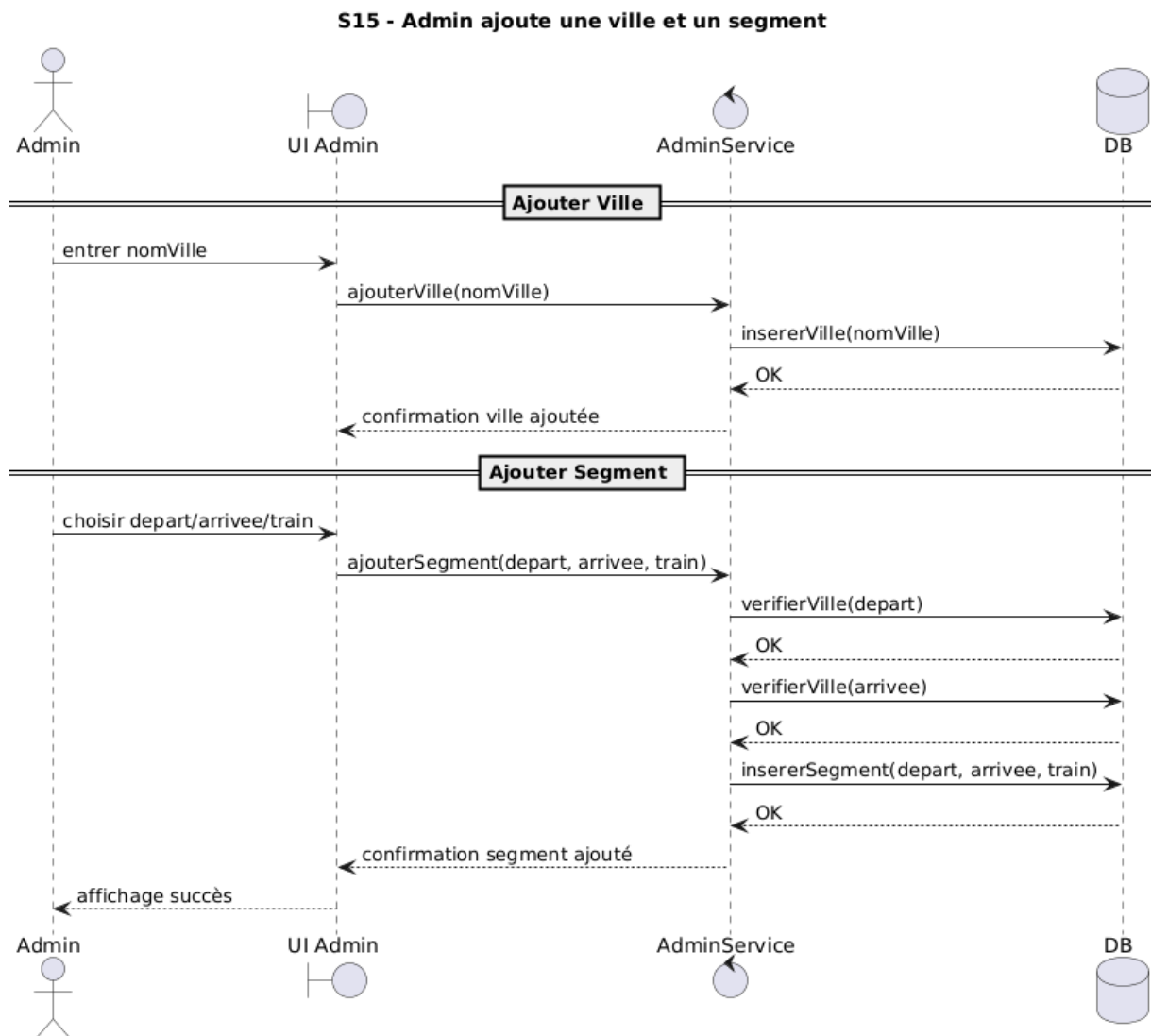
- Admin autorisé.
- Le train existe (ou est créé avant).
- Pour ajouter un segment : les villes existent.

Postconditions

- Ville ajoutée.
- Segment ajouté.
- Réseau mis à jour.

Invariants

- Un segment relie 2 villes existantes.
- Les données enregistrées doivent rester cohérentes.



6. Catalogue des questions et problèmes

I. Problèmes liés au cycle de vie du billet

Q1 — Quand un billet passe-t-il officiellement à l'état UTILISÉ ?

Le cahier précise qu'une validation acceptée entraîne un changement d'état irréversible

Mais une question se pose :

- Le billet devient-il UTILISÉ :
 - dès la première validation ?
 - ou seulement après validation du **dernier segment** ?

Problème

Dans un trajet avec correspondance :

- Si on marque le billet UTILISÉ dès le premier segment → les autres segments deviennent impossibles à valider.
- Si on attend le dernier segment → il faut gérer un état “partiellement utilisé”.

Décision d'analyse :

Le billet reste **VALIDE** tant qu'il reste des segments non validés.

Il passe à **UTILISÉ** uniquement lorsque tous les segments sont validés.

II. Problèmes liés aux correspondances

Q2 - Comment garantir la cohérence temporelle des segments ?

Le système doit vérifier que les correspondances sont compatibles.

Problèmes possibles :

- Temps de correspondance trop court
- Chevauchement d'horaires
- Ordre incohérent des segments

Décision d'analyse :

Un itinéraire est valide si :

- $\text{heureArrivée}(\text{segment } i) < \text{heureDépart}(\text{segment } i+1)$
- délai minimal de correspondance respecté

III. Problèmes liés à la concurrence

Q3 — Que se passe-t-il si deux agents scannent en même temps ?

Le cahier impose la gestion des validations simultanées .

Risque :

- Double validation acceptée
- Incohérence de l'état du billet

Décision d'analyse :

Le système doit garantir :

- Une validation atomique pour (billet, segment)
- Une seule validation acceptée
- Les autres retournent "déjà utilisé"

IV. Problèmes liés à la sécurité

Q4 — Que faire si le token est expiré ?

Le token est temporaire .

Problème :

- L'agent scanne mais son token a expiré.

Décision :

- Refuser la validation
- Obliger la ré-authentification
- Enregistrer une trace REFUSÉE

Q5 — Que faire si le QR est copié ?

Le QR contient uniquement l'UUID .

Problème :

- Quelqu'un prend une capture du QR

Réponse métier :

Même si le QR est copié :

- Seule la première validation sera acceptée
- Les suivantes seront refusées (DEJA_UTILISE)

V. Problèmes liés à la validation contextuelle

Q6 — Faut-il autoriser une tolérance horaire ?

Exemple :

- Train à 10h00
- Scan à 9h58 ou 10h05

Le cahier parle de validation selon train/date/heure
mais ne précise pas la tolérance.

Décision possible :

- Autoriser une fenêtre de $\pm X$ minutes
- Ou exiger correspondance stricte

VI. Problèmes liés à la traçabilité

Q7 — Faut-il tracer aussi les tentatives frauduleuses ?

Le cahier exige que toute validation (acceptée ou refusée) soit enregistrée.

Décision :

- Oui, toutes les tentatives sont tracées
- Inclure motif de refus

VII. Problèmes liés au périmètre

Q8 — Que faire en cas de retard ou annulation ?

Le cahier exclut explicitement :

- gestion des retards
- recalcul dynamique d'itinéraires

Donc :

Ce cas est hors périmètre du projet.

VIII. Problèmes liés aux états invalides

Q9 — Dans quels cas un billet devient INVALIDE ?

Possibilités :

- Annulation par administrateur
- Billet frauduleux détecté
- Données incohérentes

Décision :

Un billet INVALIDE :

- ne peut plus être validé
- toute tentative retourne REFUS

IX. Problèmes liés à la performance

Q10 — Comment garantir un temps de réponse < 2 secondes ?

Critère de validation .

Risques :

- Trop de requêtes
- Base de données lente

En analyse :

- Centralisation des données
- Validation rapide
- Index sur UUID

X. Problèmes liés aux données fixes

Q11 — Le réseau est fixe (≥ 10 villes)

Problème :

- Peut-on ajouter des villes dynamiquement ?

Le périmètre précise réseau fixe

Décision :

Les villes et segments sont gérés par l'administrateur.